From Conceptual Modelling to Database Schema

Modelling Hint: the Time Series Pattern

Most of your storage volume will be time series points (prices/indicators over time). In NoSQL, a reliable approach is the *time series pattern*: store many measurements for the same entity in a single, "wide" partition, ordered by time.

- Partition by the thing you read together (typically AssetID + DataSourceID).
- Cluster by time so you can efficiently read "latest" or a time range.
- When partitions get too large, shard them (for example by BusinessYear).

This pattern is an extension of the wide partition pattern used in analytics, sensor data, and finance workloads.

Conceptual Model

Start with the domain, without thinking yet about tables/collections. You should be able to answer: what is an asset, who provided the data, and what does one time series data point look like?

At minimum, capture these concepts:

- FinancialAsset: identity (AssetID), class/type (stock, FX, crypto, …), symbol, region, and optional descriptive metadata.
- DataSource: who delivered the time series (vendor/provider), plus identifiers needed for provenance and reproducibility.
- TimeSeriesPoint: a timestamped measurement that may contain multiple indicators (open, close, volume, …).

Because indicators vary by asset type and provider, treat the data of a point as flexible as possible, e.g., using a map/dictionary of k/v values where key=attribute name.

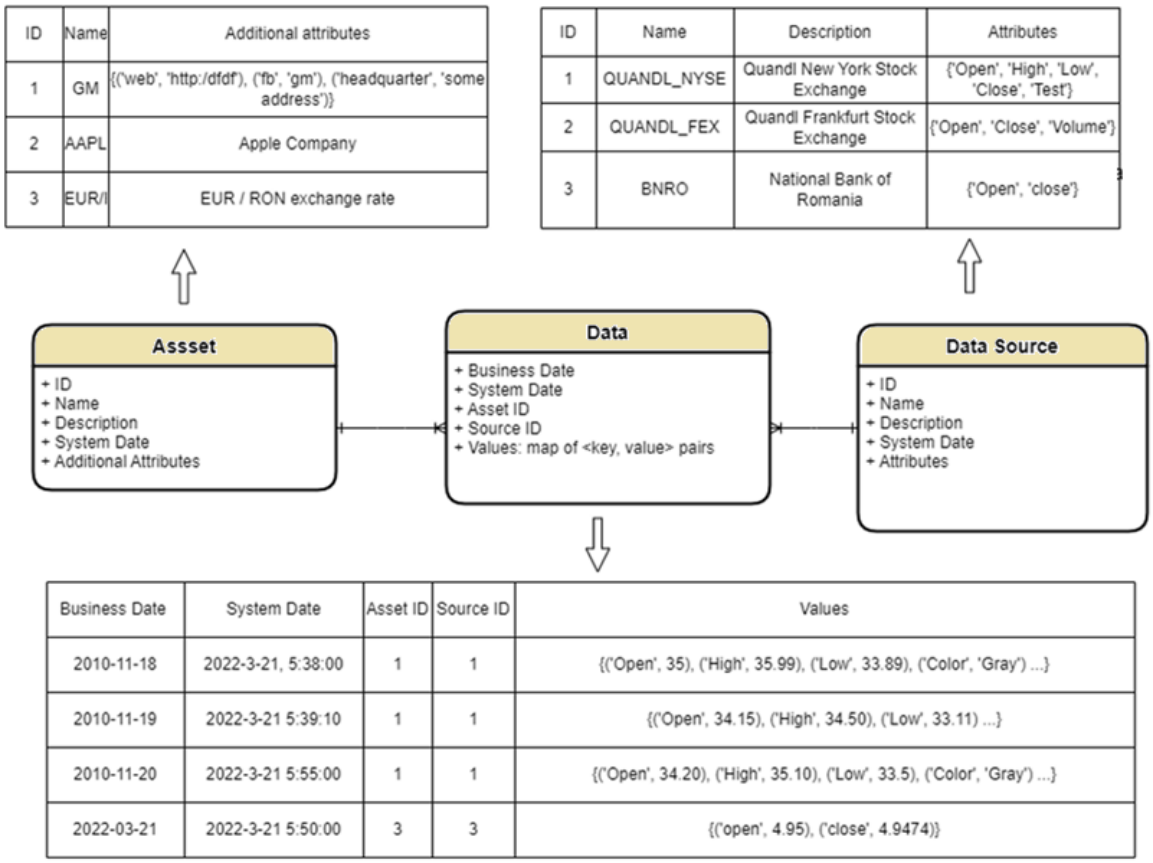

Figure 1 Proposed conceptual model (assets, data sources, and time series)

Logical Model

To build an efficient model, one should start with the frequent query patterns used in the application. The below ones are extracted from the project description: Q1 – Q5:

- How clients find assets (list assets, fetch asset details).
- How clients find data sources (list providers, fetch provider details).
- How time series are stored so [Q5] can read quickly for a given (asset, source) and time interval.
-

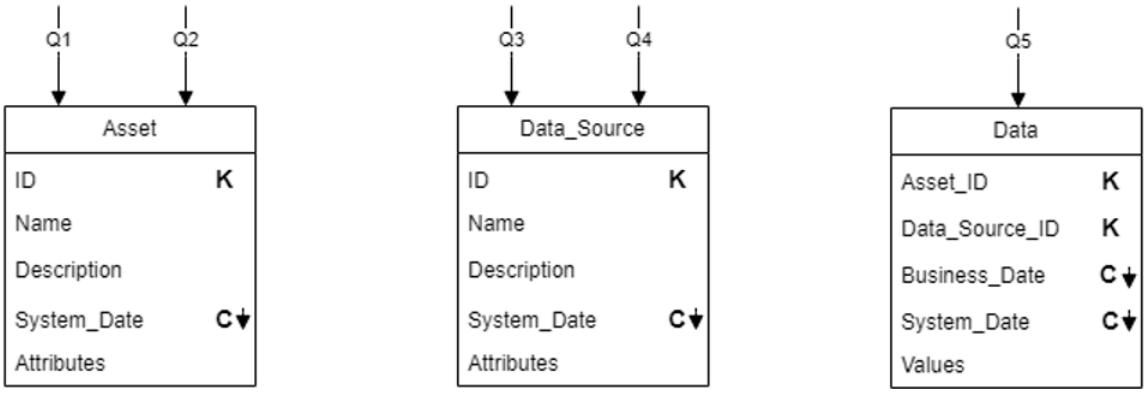

Figure 2 Proposed logical model

Key points from the model:

- Bi-temporal timestamps: Business_Date is the valid date (when the value is true in the market), and System_Date is when you stored it (audit/versioning).
- Sort System_Date in descending order so "give me the latest version" is fast.
- Avoid unbounded partitions: if a (AssetID, DataSourceID) partition grows too much, add BusinessYear (or another shard) to the partition key.

Database Schema

Finally, map the logical model to concrete tables/collections and keys. Your design must support the project's temporal rules (no in-place updates or deletes). A minimal "time series point" record should contain: the identifiers, the business timestamp, the ingestion/version timestamp, and a payload of indicators. To represent a deletion, write a special marker record (for example, payload contains deleted=true) starting from a given valid date.

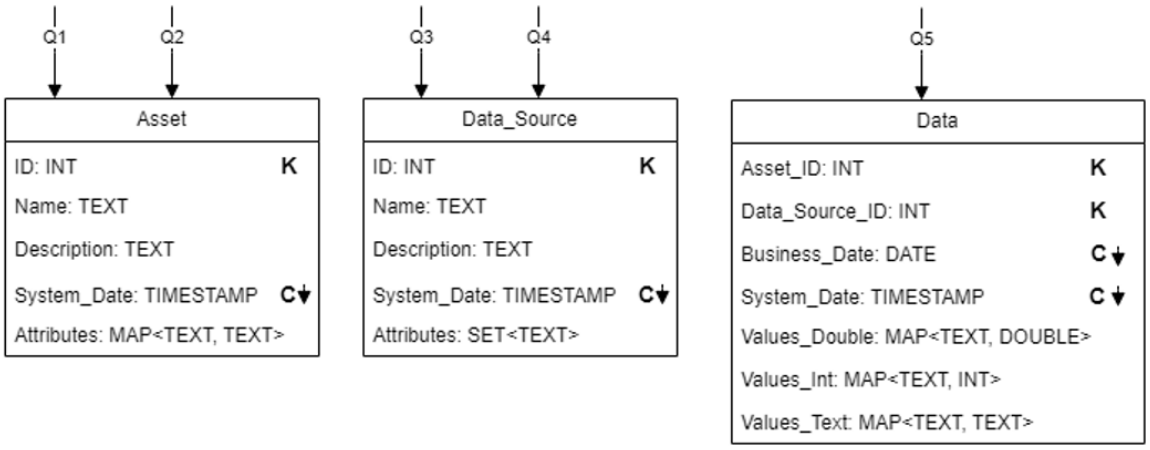

Figure 3.Proposed database schema

Design Notes and Checklist

Before you start implementing the model in a data storage system, sanity-check your model with this quick list:

- Does your schema make [Q1]-[Q5] cheap (few partitions, predictable reads)?
- Can you retrieve the latest value and also reproduce a past snapshot of the data (historical correctness), e.g., what values would have been returned by the system on 12-March-2024?
- Have you documented how you store heterogeneous indicators (map/document/attributes) and why you chose it?
- Do you have a strategy for partition growth (year sharding, bucketing, or vendor-specific limits)?
- Is provenance always stored (which provider, which dataset, which API endpoint/parameters)?

References

- Lecture 2: Data Modelling for NoSQL (see Classroom)